Labo 11 — Pourquoi Kubernetes : du Compose au cluster

Théorie — ce que Compose ne saura jamais faire, ce qu'est un cluster, pourquoi on y décrit un état au lieu de lancer des commandes, et comment k3s met tout cela dans votre WSL en une minute.

Objectifs

- Nommer les limites de Compose qui justifient un orchestrateur.
- Décrire un cluster : *control plane* et nœuds, et le rôle de chaque composant.
- Comprendre le **modèle déclaratif** et la **boucle de réconciliation**.
- Reconnaître dans le Pod Kubernetes le pod Podman du labo 07.
- Installer k3s sur WSL, dialoguer avec `kubectl`, et faire entrer une image Podman dans le cluster.

---

1. Ce que Compose ne sait pas faire

Au labo 09, `podman compose up -d` a monté la stack entière sur votre machine. Au labo 10, vous avez vu ce qu'il manquait pour la production : si la machine tombe, tout tombe ; si l'API a besoin de trois instances sur trois machines, Compose ne sait pas ; si une version doit remplacer l'autre sans couper le service, c'est à vous de le scripter ; et si un conteneur meurt, `restart` le relance — mais au même endroit, avec la même panne. Ces quatre manques ont un nom : **haute disponibilité**, **répartition**, **mise à jour progressive**, **auto-réparation**. Un orchestrateur existe pour les fournir.

> **→ HISTOIRE**
> Google fait tourner ses services dans des conteneurs depuis 2003 avec un système interne, **Borg**, qui ordonnance des millions de tâches sur des centaines de milliers de machines. En 2014, trois ingénieurs de Google (Joe Beda, Brendan Burns, Craig McLuckie) en publient une réécriture ouverte : **Kubernetes** (du grec *pilote de navire*, abrégé « k8s »). Donné en 2015 à la Cloud Native Computing Foundation, il devient en cinq ans le standard de fait des orchestrateurs, devant Swarm et Mesos. **k3s** (Rancher, 2019) est une distribution certifiée allégée pour l'edge, l'IoT et les postes de développement.

2. Un cluster : un cerveau et des bras

Un **cluster** est un ensemble de machines (*nœuds*) qui exécutent des conteneurs, pilotées par un **control plane** qui décide où et comment.

Composant	Rôle	L'analogie Podman
API server	la seule porte d'entrée : reçoit, valide et stocke chaque demande	la CLI `podman`, mais en serveur HTTPS
etcd	base clé-valeur qui conserve l'état voulu et l'état observé	la base de données locale de Podman
scheduler	choisit sur quel nœud placer chaque Pod	personne : votre machine
controller manager	les boucles qui comparent voulu et observé, et corrigent	`--restart`, mais généralisé à tout
kubelet (sur chaque nœud)	l'agent qui fait tourner les Pods demandés	`podman` + `conmon`
container runtime (containerd, via CRI)	crée réellement les conteneurs	`crun`
kube-proxy (sur chaque nœud)	le réseau des Services (labo 13)	`rootlessport`

Tout dialogue passe par l'**API server** : `kubectl`, les tableaux de bord, les contrôleurs, le kubelet lui-même. C'est ce qui rend Kubernetes extensible — tout ce qui parle HTTPS peut le piloter — et c'est pourquoi la première question d'un diagnostic est « l'API server répond-il ? » (`kubectl cluster-info`).

> **K3S**
> k3s met **tout le control plane dans un seul binaire** de 70 Mo, remplace etcd par SQLite (suffisant pour un nœud), embarque containerd, et ajoute ce qu'un cluster nu n'a pas : un Ingress controller (Traefik), un équilibreur (ServiceLB), une classe de stockage (`local-path`), CoreDNS et metrics-server. Il tourne en **root**, comme un service systemd — c'est le kubelet qui crée les conteneurs, pas vous. Un cluster d'entreprise sépare ces composants sur plusieurs machines ; les commandes et les manifestes sont **les mêmes**.

3. Le modèle déclaratif

Avec Podman, vous donnez des **ordres** : `run`, `stop`, `rm`. Avec Kubernetes, vous déposez une **description** : « je veux un Pod nommé `api`, avec cette image, ce port, cette variable ». L'API server la stocke ; les contrôleurs et le kubelet travaillent jusqu'à ce que la réalité y corresponde ; et ils **continuent** de surveiller. Si le Pod disparaît, l'écart entre voulu et observé est détecté et corrigé. C'est la **boucle de réconciliation**, le cœur de tout Kubernetes.

Un objet se décrit en YAML avec quatre clés : `apiVersion` (le groupe et la version de l'API), `kind` (le type : Pod, Deployment, Service…), `metadata` (nom, namespace, labels), `spec` (ce que vous voulez). Le cluster y ajoute `status` (ce qu'il observe). `kubectl apply -f fichier.yaml` envoie la spec ; `kubectl get pod api -o yaml` rend spec **et** status.

> **À RETENIR**
> `kubectl apply` est **idempotent** : relancé dix fois, il ne change rien si le fichier n'a pas changé, et il met à jour si le fichier a changé. `kubectl create` crée une fois et échoue la seconde (`AlreadyExists`). Un manifeste versionné + `apply` = la même discipline que Compose, mais appliquée par un système qui ne s'arrête jamais de vérifier.

Au labo 11, le Pod seul est encore un objet **sans contrôleur** : supprimé, il ne revient pas. C'est le Deployment (labo 12) qui porte la boucle « je veux N réplicas ». Vous le verrez à la fin du labo pratique : un Pod effacé sous un Deployment réapparaît en quelques secondes.

4. Le Pod

L'unité de déploiement n'est pas le conteneur mais le **Pod** : un ou plusieurs conteneurs qui partagent une adresse IP, un `localhost`, et des volumes. Vous en avez déjà fait un au labo 07 avec `podman pod create` : même concept, même mot, même `/etc/hosts` partagé. Un Pod reçoit une IP du réseau du cluster (`10.42.0.x` chez k3s) ; elle change à chaque recréation, et aucun client ne doit la coder en dur — ce sera le rôle des Services (labo 13).

> **→ RAPPEL DOCKER**
> Un conteneur Podman = un processus isolé ; un pod Podman = plusieurs conteneurs dans le même network namespace ; `podman generate kube` écrit ce pod en YAML Kubernetes et `podman kube play` le relit. Le Pod Kubernetes est exactement cet objet, placé par un scheduler sur un nœud et surveillé par un kubelet. Ce que vous saviez des conteneurs reste vrai à l'intérieur : PID 1, signaux, codes de sortie, logs sur stdout, variables d'environnement.

> **→ RÉSEAU**
> Chaque nœud reçoit une plage d'adresses pour ses Pods (`10.42.0.0/24` sur le premier nœud k3s), et un **CNI** (*Container Network Interface*, ici flannel) relie les plages des nœuds entre elles : tout Pod joint tout Pod par son IP, sans NAT. C'est le réseau user-defined du labo 07 étendu à plusieurs machines. L'hôte, lui, garde son adresse (`hostIP` dans le status du Pod).

Dans la pratique, la plupart des Pods n'ont **qu'un** conteneur ; le second est un *sidecar* (agent de logs, proxy TLS) ou un *init container* (migration de schéma avant l'API).

5. `kubectl` et le kubeconfig

`kubectl` lit un fichier **kubeconfig** (`~/.kube/config`, ou `KUBECONFIG`, ou pour k3s `/etc/rancher/k3s/k3s.yaml`) qui contient l'adresse de l'API server, le certificat de l'autorité, et vos identifiants ; un **contexte** associe un cluster, un utilisateur et un **namespace** par défaut. Les namespaces cloisonnent les objets ; dans cette série, chaque labo travaille dans le sien (`kubectl create namespace lab11`, puis `kubectl config set-context --current --namespace=lab11`).

Commande	Usage
`kubectl get pods` (`-o wide`, `-o yaml`, `-A`)	lister, avec le nœud et l'IP, ou l'objet complet, ou tous les namespaces
`kubectl describe pod api`	détails + **Events** : la chronologie de ce que le cluster a fait
`kubectl logs api`, `kubectl exec api -- cmd`	comme `podman logs` / `podman exec`
`kubectl apply -f`, `kubectl delete -f`	déposer / retirer des manifestes
`kubectl run web --image=…`	créer un Pod sans fichier (test rapide)
`kubectl port-forward pod/api 18080:8080`	tunnel local vers un Pod, sans publier de port
`kubectl explain pod.spec.containers`	la documentation de chaque champ, hors ligne
`kubectl api-resources`	tous les types d'objets du cluster

→ WINDOWS / WSL
Installation en une ligne, dans votre Ubuntu : `curl -sfL https://get.k3s.io | sudo sh -s - --write-kubeconfig-mode 644`. Le script installe le binaire, un service systemd `k3s` (donc `systemd=true` est obligatoire) et un `kubectl`. `--write-kubeconfig-mode 644` rend le kubeconfig lisible sans `sudo` : commode sur un poste, inacceptable sur un serveur partagé (ce fichier **est** l'accès administrateur au cluster). Comptez 4 Go de RAM pour WSL (`.wslconfig`, `memory=6GB` conseillé) ; `wsl --shutdown` arrête le cluster, systemd le relance au prochain `wsl`.

6. Faire entrer une image dans le cluster

Le cluster a **son propre** magasin d'images, celui du containerd de k3s (en root, dans `/var/lib/rancher/k3s`). Podman a le sien (`~/.local/share/containers`). Un `podman build` ne donne donc rien au cluster : `kubectl apply` d'un Pod sur `localhost/api:1.0` finit en `ErrImagePull` puis `ImagePullBackOff`, car le kubelet tente de **tirer** l'image depuis un registry nommé `localhost`. Deux solutions : l'importer (`podman save localhost/api:1.0 | sudo k3s ctr images import -`), ce que fait cette série ; ou la pousser dans un registry que le cluster connaît (labo 17). Dans les deux cas, `imagePullPolicy: IfNotPresent` dit au kubelet d'utiliser l'image locale si elle existe ; `Always` (le défaut pour le tag `latest`) forcerait un pull à chaque démarrage.

PODMAN
Podman parle Kubernetes nativement : `podman generate kube` écrit vos pods en YAML, `podman kube play` relit un manifeste Kubernetes et crée les pods correspondants sans cluster. Le fichier produit contient toutefois des choses propres à une machine (`hostPath`, `hostPort`) qu'un cluster remplacera par des ConfigMaps et des Services. Et `sudo k3s crictl ps` / `sudo k3s crictl images` montrent les conteneurs et images **du cluster**, là où `podman ps` ne montre que les vôtres.

7. En entreprise

- **Le développeur écrit des manifestes, l'équipe plateforme fait tourner le cluster.** Vous recevrez un kubeconfig avec un contexte par environnement et un namespace par équipe ; vous ne verrez jamais le control plane.
- **Un cluster managé** (EKS, AKS, GKE) ou OpenShift remplace k3s en production : control plane géré par le fournisseur, nœuds à la demande, équilibreurs et stockage branchés sur le cloud. Vos manifestes, eux, ne changent pas.
- **Compose reste sur le poste** ; Kubernetes commence au premier environnement partagé. Beaucoup d'équipes gardent les deux, et un `kubectl apply` fait en CI ce que `compose up` fait sur le poste (labo 17, GitOps).
- **Le vocabulaire compte** : Pod, Deployment, Service, Ingress, namespace, manifeste, réconciliation. C'est celui des incidents, des revues et des entretiens.

À retenir

- Compose ne fait ni haute disponibilité, ni répartition, ni mise à jour progressive, ni auto-réparation : c'est le travail d'un orchestrateur.
- Un cluster = un control plane (API server, etcd, scheduler, controllers) + des nœuds (kubelet, containerd, kube-proxy) ; tout passe par l'API server.
- Kubernetes est **déclaratif** : on dépose un état voulu, des boucles de réconciliation le maintiennent ; `apply` est idempotent.

- Le Pod est l'unité de déploiement — le pod Podman du labo 07 — avec une IP éphémère du réseau du cluster.
- `kubectl` lit un kubeconfig, travaille dans un contexte et un namespace ; `describe` et ses Events sont le premier outil de diagnostic.
- Le cluster a son propre magasin d'images : `podman save | sudo k3s ctr images import -`, et `imagePullPolicy: IfNotPresent`.
- k3s = tout Kubernetes dans un binaire, avec Traefik, ServiceLB et `local-path` ; les manifestes sont ceux de n'importe quel cluster.

Vocabulaire

orchestrateur : système qui place, surveille et répare des conteneurs sur plusieurs machines. — **cluster / nœud** : ensemble de machines / une machine qui exécute des Pods. — **control plane** : API server, etcd, scheduler, controller manager. — **kubelet** : agent de nœud qui fait tourner les Pods. — **CRI / containerd** : interface et runtime de conteneurs du nœud. — **manifeste** : fichier YAML décrivant un objet (`apiVersion`, `kind`, `metadata`, `spec`). — **état voulu / observé** : `spec` / `status`. — **boucle de réconciliation** : contrôleur qui corrige l'écart entre les deux. — **Pod** : groupe de conteneurs partageant IP, `localhost` et volumes. — **CNI** : plugin réseau qui relie les Pods entre nœuds. — **kubeconfig / contexte / namespace** : accès au cluster / cluster + utilisateur + namespace par défaut / cloison d'objets. — **`imagePullPolicy`** : quand le kubelet tire une image. — **k3s** : distribution Kubernetes certifiée en un binaire.